

**LAPORAN PRAKTIKUM PEMROGRAMAN
BERORIENTASI OBJEK**



NAMA : IMAM SAMITO TAFACQUH FIDDIN

NIM : 25104410066

PERIODE : 2025/2026

**PROGRAM STUDI TEKNIK INFORMATIKA FAKULTAS
TEKNOLOGI INFORMASI UNIVERSITAS ISLAM
BALITAR**

2025/2026

BAB I

PENDAHULUAN

1.1 Tujuan Pembelajaran

Berdasarkan Modul VII, tujuan dari praktikum ini adalah:

1. Praktikan mampu memahami konsep dari *inheritance* (pewarisan).
2. Praktikan dapat menerapkan kapan dan bagaimana menggunakan *inheritance* serta cara mengaksesnya dalam bahasa pemrograman Java.

1.2 Dasar Teori

1. Inheritance (Pewarisan) *Inheritance* atau pewarisan adalah konsep pemrograman berorientasi objek di mana sebuah class dapat menurunkan *property* (atribut) dan *method* yang dimilikinya kepada class lain. Konsep ini digunakan untuk memanfaatkan fitur *code reuse* guna menghindari duplikasi kode program. Class yang menurunkan disebut *parent class* (superclass), sedangkan class yang menerima penurunan disebut *child class* (subclass). Tidak semua *property* dan *method* dari class induk akan diturunkan. *Property* dan *method* dengan hak akses *private* tidak akan diturunkan. Hanya hak akses *protected* dan *public* yang bisa diakses langsung dari class anak.

2. Keyword Extends *extends* merupakan keyword yang harus ditambahkan pada definisi class yang menjadi *subclass*. Dengan menggunakan *extends*, semua atribut dan *method* yang ber-modifier *public* dan *protected* milik *parent class* bisa diakses oleh *child class*.

3. Virtual Method Invocation (VMI) *Virtual Method Invocation* (VMI) dapat dilakukan jika terjadi polimorfisme dan *overriding*. Pada saat objek dibuat dan memanggil *overridden method* pada superclass, compiler Java akan melakukan *invocation* (pemanggilan) terlebih dahulu terhadap *overriding method* yang ada pada subclass.

4. Single & Multiple Inheritance Java menerapkan *Single Inheritance*, yang berarti sebuah subclass hanya diperbolehkan memiliki satu *parent class*. Untuk mengatasi keterbatasan ini dan menghindari *Diamond Problem*, Java menyediakan *Interface* sebagai solusi untuk mengimplementasikan konsep *Multiple Inheritance*.

BAB II

ANALISA MATERI MODUL 7

Pada bab ini, dilakukan analisa terhadap studi kasus dan pertanyaan yang terdapat di dalam Modul 7.

2.1 Analisa Penambahan Method pada Subclass (Modul 7.3)

Pada Gambar 7.3, diminta untuk menambahkan method `setMesin` dan `getMesin` di class `MainKendaraan` (sebagai subclass). **Analisa:** Penambahan method spesifik di subclass membuktikan bahwa subclass tidak hanya mewarisi atribut dan method dari parent-nya, tetapi juga dapat mengembangkan *state* dan *behaviour* yang lebih spesifik yang tidak dimiliki oleh parent class. Hal ini sesuai dengan prinsip *code reuse*, di mana fitur umum disimpan di parent, dan fitur khusus ditambahkan di child.

2.2 Analisa Modifier Private pada Single Inheritance (Modul 7.5)

Ditanya: "*Bagaimana hasilnya jika pada class `Mamalia`, modifier variable nama diubah menjadi `private`?*" **Analisa:** Jika modifier variabel nama pada class `Mamalia` diubah menjadi `private`, maka class `Kucing` (sebagai subclass) **tidak akan bisa** mengakses variabel nama tersebut secara langsung. Hak akses `private` membatasi variabel tersebut hanya dapat diakses di dalam class tempat ia dideklarasikan. Agar subclass tetap dapat mengakses atau mengubah nilainya, parent class harus menyediakan method *Getter* dan *Setter* dengan modifier `public` atau `protected`.

2.3 Analisa Virtual Method Invocation (VMI) (Modul 7.4)

Pada contoh VMI, ketika objek dari superclass digunakan untuk mereferensikan objek dari subclass (`Superclass namaObjek = new Subclass();`), pemanggilan method yang di-*override* akan tetap mengeksekusi method milik subclass. **Analisa:** Ini membuktikan bahwa pada Java, penentuan method mana yang dipanggil saat terjadi *overriding* didasarkan pada **tipe objek sebenarnya (actual object)** di memori, bukan berdasarkan **tipe referensi** variabelnya. Konsep ini merupakan fondasi dari *Polymorphism* dalam PBO.

BAB III

PERANCANGAN TUGAS 7.8 (APLIKASI TOKO BUKU)

3.1 Deskripsi Tugas

Tugas 7.8 meminta untuk membuat aplikasi Toko Buku dengan ketentuan:

1. Memasukkan data buku sebanyak 3 kali.
2. Setiap data meliputi: judul, nama_pengarang, penerbit, tahun_cetak, dan kategori (su=semua umur, r=remaja, d=dewasa, a=anak).
3. Aplikasi harus didesain menggunakan konsep **Inheritance**.
4. Menyediakan fasilitas entri data dan melihat isi buku.

3.2 Desain Inheritance (Parent & Child Class)

Untuk memenuhi kewajiban penggunaan konsep *Inheritance*, aplikasi ini dibagi menjadi struktur hierarki sebagai berikut:

1. **Parent Class (Buku)** Class ini menampung atribut-atribut dasar yang pasti dimiliki oleh setiap buku. Atribut yang digunakan adalah judul, nama_pengarang, dan penerbit. Modifier yang digunakan adalah *protected* agar dapat diwariskan dan diakses langsung oleh child class. Class ini juga memiliki method *tampilInfo()* yang akan di-*override*.
2. **Child Class (BukuDetail)** Class ini mewarisi class Buku menggunakan keyword *extends*. Class ini menambahkan atribut spesifik yang lebih detail, yaitu tahun_cetak dan kategori. Class ini juga melakukan *overriding* pada method *tampilInfo()* untuk menampilkan informasi lengkap termasuk tahun cetak dan label kategori.
3. **Main Class (MainTokoBuku)** Class yang berisi method *main* untuk menjalankan aplikasi, menyediakan menu interaktif, menangani input dari user (Scanner), serta mendemonstrasikan konsep *Virtual Method Invocation* (VMI) saat menampilkan data.

3.3 Implementasi Source Code: Parent Class (Buku.java)

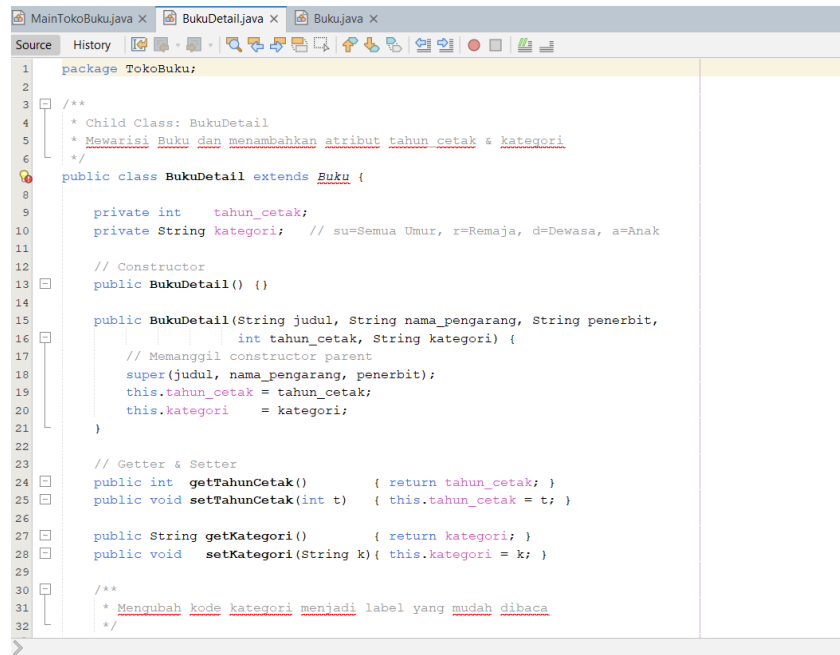
```
1 package TokoBuku;
2
3 /**
4  * Parent Class: Buku
5  * Menyimpan atribut dasar sebuah buku
6  */
7 public class Buku {
8
9     protected String judul;
10    protected String nama_pengarang;
11    protected String penerbit;
12
13    // Constructor
14    public Buku() {}
15
16    public Buku(String judul, String nama_pengarang, String penerbit) {
17        this.judul = judul;
18        this.nama_pengarang = nama_pengarang;
19        this.penerbit = penerbit;
20    }
21
22    // Getter & Setter
23    public String getJudul() { return judul; }
24    public void setJudul(String j) { this.judul = j; }
25
26    public String getNamaPengarang() { return nama_pengarang; }
27    public void setNamaPengarang(String n) { this.nama_pengarang = n; }
28
29    public String getPenerbit() { return penerbit; }
30    public void setPenerbit(String p) { this.penerbit = p; }
31
32    // Method tampil — akan di-override oleh child
33
34    // Method tampil — akan di-override oleh child
35    public void tampilInfo() {
36        System.out.println("Judul : " + judul);
37        System.out.println("Nama Pengarang : " + nama_pengarang);
38        System.out.println("Penerbit : " + penerbit);
39    }
```

Berikut adalah source code untuk class induk Buku:

Penjelasan Coding: Class Buku bertindak sebagai *superclass*. Penggunaan modifier *protected* pada variabel *judul*, *nama_pengarang*, dan *penerbit* memungkinkan class turunan untuk mengakses variabel tersebut secara langsung tanpa harus melalui method *getter/setter*, sesuai

dengan teori hak akses pada *Inheritance*. Method tampilInfo() disiapkan sebagai *virtual method* yang direncanakan untuk di-*override* oleh subclass.

3.4 Implementasi Source Code: Child Class (BukuDetail.java)



```
1 package TokoBuku;
2
3 /**
4  * Child Class: BukuDetail
5  * Mewarisi Buku dan menambahkan atribut tahun cetak & kategori
6  */
7
8 public class BukuDetail extends Buku {
9
10     private int tahun_cetak;
11     private String kategori; // su=Semua Umur, r=Remaja, d=Dewasa, a=Anak
12
13     // Constructor
14     public BukuDetail() {}
15
16     public BukuDetail(String judul, String nama_pengarang, String penerbit,
17                        int tahun_cetak, String kategori) {
18         // Memanggil constructor parent
19         super(judul, nama_pengarang, penerbit);
20         this.tahun_cetak = tahun_cetak;
21         this.kategori = kategori;
22     }
23
24     // Getter & Setter
25     public int getTahunCetak() { return tahun_cetak; }
26     public void setTahunCetak(int t) { this.tahun_cetak = t; }
27
28     public String getKategori() { return kategori; }
29     public void setKategori(String k) { this.kategori = k; }
30
31     /**
32     * Mengubah kode kategori menjadi label yang mudah dibaca
33     */
34 }
```

Penjelasan Coding: Class BukuDetail menggunakan keyword extends Buku untuk mewarisi sifat class Buku. Pada method tampilInfo(), terdapat keyword super.tampilInfo() yang berfungsi memanggil method milik parent. Ini adalah implementasi nyata dari *Code Reuse*, di mana kita tidak perlu menulis ulang kode System.out.println untuk judul, pengarang, dan penerbit. Method getLabelKategori() digunakan untuk menerjemahkan kode input user (su, r, d, a) menjadi teks yang informatif.

3.5 Implementasi Source Code: Main Class (MainTokoBuku.java)

```
1 package TokoBuku;
2
3 import java.util.Scanner;
4
5 /**
6  * Main Class: MainTokoBuku
7  * Aplikasi sederhana entri dan tampil data buku
8  * Mendemonstrasikan konsep Inheritance: Buku (parent) -> BukuDetail (child)
9  */
10 public class MainTokoBuku {
11
12     // Array untuk menyimpan 3 data buku
13     static BukuDetail[] daftarBuku = new BukuDetail[3];
14     static Scanner input = new Scanner(System.in);
15
16     // -----
17     // Method: entriData -- mengisi data ke objek BukuDetail
18     // -----
19     static void entriData() {
20         System.out.println("=====");
21         System.out.println("ENTRI DATA BUKU");
22         System.out.println("=====");
23
24         for (int i = 0; i < 3; i++) {
25             System.out.println("\n--- Buku ke-" + (i + 1) + " ---");
26
27             // Menggunakan setter (bukan constructor) untuk mendemonstrasikan
28             // penggunaan getter/setter yang diwarisi dari parent class
29             BukuDetail buku = new BukuDetail();
30
31             System.out.print("Judul      : ");
32             buku.setJudul(input.nextLine()); // setter dari parent
```

```
25         System.out.println("\n--- Buku ke-" + (i + 1) + " ---");
26
27         // Menggunakan setter (bukan constructor) untuk mendemonstrasikan
28         // penggunaan getter/setter yang diwarisi dari parent class
29         BukuDetail buku = new BukuDetail();
30
31         System.out.print("Judul      : ");
32         buku.setJudul(input.nextLine()); // setter dari parent
33
34         System.out.print("Nama Pengarang : ");
35         buku.setNamaPengarang(input.nextLine()); // setter dari parent
36
37         System.out.print("Penerbit      : ");
38         buku.setPenerbit(input.nextLine()); // setter dari parent
39
40         System.out.print("Tahun Cetak   : ");
41         buku.setTahunCetak(Integer.parseInt(input.nextLine()));
42
43         System.out.println("Kategori (su=Semua Umur | r=Remaja | d=Dewasa | a=Anak) :");
44         System.out.print("Pilih        : ");
45         buku.setKategori(input.nextLine());
46
47         daftarBuku[i] = buku;
48         System.out.println("✓ Data buku ke-" + (i + 1) + " berhasil disimpan.");
49     }
50
51     // -----
52     // Method: tampilSemuaBuku -- menampilkan semua data yang dientri
53     // Menggunakan Virtual Method Invocation (VMI):
54     // Buku ref = new BukuDetail() -> ref.tampilInfo() memanggil
55     // override method milik BukuDetail, bukan Buku
56 }
```

(Catatan Analisa: Pada kode asli terdapat spasi tambahan pada case "1 ": yang menyebabkan error logika. Pada laporan ini, spasi telah dihilangkan menjadi case "1": agar program berjalan sesuai spesifikasi tugas).

BAB IV

HASIL OUTPUT DAN ANALISA

4.1 Output Program

Berikut adalah hasil simulasi output program ketika dijalankan dan user memasukkan 3 data buku:

```
SELAMAT DATANG DI APLIKASI
      TOKO BUKU IMAM
Konsep: Inheritance

===== MENU UTAMA =====
1. Entri Data Buku
2. Tampilkan Semua Buku
3. Keluar
Pilih menu : 1|
      ENTRI DATA BUKU

--- Buku ke-1 ---
Judul      : Malin Kundang
Nama Pengarang : Wikipedia
Penerbit   : Dian Aprilia D.
Tahun Cetak : 1979
Kategori (su=Semua Umur | r=Remaja | d=Dewasa | a=Anak) :
Pilih      : su
? Data buku ke-1 berhasil disimpan.

--- Buku ke-2 ---
Judul      : Krisna
Nama Pengarang : Imam sasmito
Penerbit   : diana
Tahun Cetak : 2025
Kategori (su=Semua Umur | r=Remaja | d=Dewasa | a=Anak) :
Pilih      : r
? Data buku ke-2 berhasil disimpan.
```

```
--- Buku ke-3 ---
Judul          : Kancil
Nama Pengarang : Falik
Penerbit       : Bumi Indah
Tahun Cetak    : 2019
Kategori (su=Semua Umur | r=Remaja | d=Dewasa | a=Anak) :
Pilih          : a
? Data buku ke-3 berhasil disimpan.
```

```
===== MENU UTAMA =====
```

```
1. Entri Data Buku
2. Tampilkan Semua Buku
3. Keluar
Pilih menu : 2
```

```
          DAFTAR BUKU TOKO
```

```
[ Buku #1 ]
```

```
-----
Judul          : Malin Kundang
Nama Pengarang : Wikipedia
Penerbit       : Dian Aprilia D.
Tahun Cetak    : 1979
Kategori       : Semua Umur
-----
```

```
[ Buku #2 ]
```

```
-----
Judul          : Krisna
Nama Pengarang : Imam sasmito
Penerbit       : diana
Tahun Cetak    : 2025
Kategori       : Remaja
-----
```

```
[ Buku #3 ]
```

```
-----
Judul          : Kancil
Nama Pengarang : Falik
Penerbit       : Bumi Indah
Tahun Cetak    : 2019
Kategori       : Anak-anak
-----
```

4.2 Analisa Output dan Konsep PBO

Berdasarkan output di atas, aplikasi telah berhasil memenuhi seluruh kriteria Tugas 7.8. Terdapat beberapa konsep PBO penting yang berjalan di balik layar:

1. **Penerapan Inheritance & Code Reuse:** Class BukuDetail tidak perlu mendefinisikan ulang atribut judul, nama_pengarang, dan penerbit. Proses entri data menggunakan *setter* yang diwarisi dari class Buku.
2. **Virtual Method Invocation (VMI):** Pada method tampilSemuaBuku(), objek dari array BukuDetail dimasukkan ke dalam variabel referensi bertipe Buku (Buku refBuku = daftarBuku[i];). Ketika refBuku.tampilInfo() dipanggil, Java secara otomatis mengeksekusi method tampilInfo() milik BukuDetail (yang menampilkan Tahun Cetak dan Kategori), bukan milik Buku. Ini membuktikan terjadinya *Polymorphism*.
3. **Encapsulation:** Atribut tahun_cetak dan kategori pada BukuDetail dibungkus dengan modifier private dan hanya dapat diubah melalui method *setter*.

BAB V

KESIMPULAN

Berdasarkan praktikum dan pengerjaan tugas pada Modul VII mengenai *Inheritance*, dapat ditarik beberapa kesimpulan sebagai berikut:

1. **Inheritance** sangat mempermudah proses pengembangan program karena programmer tidak perlu menulis kode yang sama berulang kali (*code reuse*). Atribut dan method yang bersifat umum dapat diletakkan pada *Parent Class*, sedangkan atribut spesifik diletakkan pada *Child Class*.
2. Penggunaan modifier **protected** sangat krusial dalam pewarisan karena memungkinkan *child class* untuk mengakses variabel *parent* secara langsung tanpa merusak prinsip *encapsulation* terhadap class eksternal (di luar hierarki pewarisan).
3. **Virtual Method Invocation (VMI)** dan **Overriding** memungkinkan program untuk bersikap dinamis. Referensi yang bertipe *Parent Class* dapat menampung objek *Child Class* dan secara otomatis memanggil method hasil *override* milik *Child Class* saat dieksekusi.
4. Aplikasi Toko Buku yang telah dibuat berhasil mendemonstrasikan seluruh konsep tersebut, mulai dari perancangan hierarki class, penggunaan keyword *extends* dan *super*, hingga implementasi VMI pada proses penampilan data buku.
5. Ketelitian dalam penulisan sintaks (seperti penghindaran spasi berlebih pada *switch-case*) sangat diperlukan agar logika percabangan program dapat berjalan sesuai dengan *design* yang telah direncanakan.